

Real-world (config + logging together) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: its own folder, `practice/config_logging_wiring/`, because it needs more than one file:

```
practice/config_logging_wiring/
├─ exceptions.py      MissingConfigError
├─ config.py          load_config() -> Config
├─ logging_setup.py   get_logger(name)
├─ main.py            wires the two together
└─ .env              you create this; never committed
```

Run it: `cd practice/config_logging_wiring && python main.py` — from inside the folder, so from `config` import `load_config` finds the file sitting next to it.

Builds on: `practice/logging_practice.py` ([Intermediate part 2](#)) — its `get_logger()` is **copied** into `logging_setup.py` below, unchanged. And `practice/custom_errors_practice.py` ([Basic](#)) — the same custom-error pattern, re-written here as `MissingConfigError`. Copies, not imports: this folder runs on its own.

How the approaches below are organised: all four files are shown complete in **Basic Approach 1**, in the order you create them. Every later approach changes `main.py` only, so each one shows `main.py` in full and names the files you keep exactly as they are.

Basic Version

Approach 1 — the direct way

The file you create first, because nothing runs without it — your own `.env`, holding a fake key for now:

```
# practice/config_logging_wiring/.env
OPENAI_API_KEY=sk-test-123
LOG_LEVEL=DEBUG
```

Then the error class — one line of real content, the same pattern as the Basic exercise:

```
# practice/config_logging_wiring/exceptions.py
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

Then the config loader:

```
# practice/config_logging_wiring/config.py
import os
from dataclasses import dataclass

from dotenv import load_dotenv

from exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value

def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

Then the logger — this is `get_logger()` copied straight out of `practice/logging_practice.py`, name and signature untouched:

```
# practice/config_logging_wiring/logging_setup.py
# Copied from practice/logging_practice.py (Intermediate part 2), unchanged.
import logging

def get_logger(name: str) -> logging.Logger:
    """Create (or reuse) a logger with a screen handler and a file handler.
```

```

Safe to call more than once – if this logger already has handlers,
they are not added again.
"""
logger = logging.getLogger(name)
if logger.handlers:
    return logger

logger.setLevel(logging.DEBUG)

screen_handler = logging.StreamHandler()
screen_handler.setLevel(logging.INFO)
logger.addHandler(screen_handler)

file_handler = logging.FileHandler("app.log")
file_handler.setLevel(logging.DEBUG)
logger.addHandler(file_handler)

return logger

```

And last, the file this exercise is really about — the wiring:

```

# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger("main")

logger.info("Loaded config with log level: " + config.log_level)

```

Expected output (terminal), with the `.env` above:

```
Loaded config with log level: DEBUG
```

The exact text depends on what's actually in your `.env` — this shows the shape of the output, not a fixed value. The copied `get_logger()` also attaches a file handler, so the same line is appended to `practice/config_logging_wiring/app.log`; that file appearing after your first run is expected, not a mistake. If `load_config()` fails (a required setting is missing), this version crashes with a raw Python traceback — fine for a first working version, but not what you'd want a user to see.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Both approaches below keep `practice/config_logging_wiring/exceptions.py`, `config.py` and `logging_setup.py` exactly as shown in Basic Approach 1 above — copy those three files as they are. Only `main.py` changes.

Approach 1 — f-string, no failure handling yet

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal), with the `.env` from Basic Approach 1:

```
Loaded config with log level: DEBUG
```

Approach 2 — with the startup failure handled

If config loading can fail, catch that and stop the program with a clear message instead of a scary error dump.

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    print(f"Startup failed: {e}")
    sys.exit(1)
```

```
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output when config is valid: identical to Approach 1.

Expected output when `OPENAI_API_KEY` is missing from `.env` (or set to an empty string):

```
Startup failed: Required environment variable is missing: OPENAI_API_KEY
```

and the process exits with status code `1` — no traceback, no logger involved (there isn't one yet at this point).

Difference from Basic: both Intermediate approaches switch to an f-string and `get_logger(__name__)` (so the logger is named after whichever module actually created it, not a hardcoded `"main"`). Approach 2 additionally wraps the one call that can fail — `load_config()` — in a `try / except`, so a missing setting produces one clear line and a clean exit instead of a multi-line traceback a non-technical user would have to decode.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

All three approaches below still keep `practice/config_logging_wiring/exceptions.py`, `config.py` and `logging_setup.py` exactly as shown in Basic Approach 1 above — copy those three files as they are. Only `main.py` changes.

Approach 1 — the minimal version, as a baseline

Shown again here so all 3 Advanced approaches sit side by side — this is the same wiring as Intermediate Approach 1, with no failure handling yet, included as the baseline the next two approaches build on.

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal), with the `.env` from Basic Approach 1:

```
Loaded config with log level: DEBUG
```

Approach 2 — clean startup-failure handling, one line

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    sys.exit(f"Startup failed: {e}")
```

```
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output when `OPENAI_API_KEY` is missing from `.env` :

```
Startup failed: Required environment variable is missing: OPENAI_API_KEY
```

printed to stderr, process exits with status `1` — same user-visible result as Intermediate Approach 2's `print()` + `sys.exit(1)` pair, but written as one line: `sys.exit(some_string)` prints that string to stderr and exits with code `1` automatically.

Approach 3 — wrapped in `main()` , with the startup event itself logged

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        sys.exit(f"Startup failed: {e}")

    logger = get_logger(__name__)
    logger.info(
        "startup complete",
        extra={"log_level": config.log_level, "event": "startup"},
    )

if __name__ == "__main__":
    main()
```

Expected output (terminal) — the copied `get_logger()` attaches a plain handler with no custom formatter, so the line is the bare message:

```
startup complete
```

Note what this does *not* show: the `extra={"log_level": ..., "event": "startup"}` values don't appear in that line. `extra=` attaches those fields to the log record itself, not to the printed text — a plain formatter only prints `%(message)s`, so it ignores them. They're there for a formatter (or a log-processing tool) that's written to read them, typically a JSON log formatter that would output something like `{"message": "startup complete", "log_level": "DEBUG", "event": "startup", ...}` instead of a plain sentence. This is what "structured logging" means in practice: the data exists as real fields a machine can filter on, not just as words inside a sentence a human has to parse.

Difference from Intermediate, and between these 3 Advanced approaches:

Intermediate Approach 2 already handles the failure path correctly with `print() + sys.exit(1)` — that's genuinely fine to ship. Approach 1 here is the same happy-path wiring shown again as a baseline. Approach 2 tightens the failure path to Python's `sys.exit(message)` idiom — same behavior, one line instead of two. Approach 3 is the real jump: wrapping startup in `main()` behind `if __name__ == "__main__":` means this file becomes safely importable elsewhere (its top-level code doesn't run on import), and logging a "startup complete" event — with structured `extra=` data — means a successful start leaves a trace too, not just failures. That trace is what you'd search for in production logs to answer "was this instance actually running with the config we expected?"

Which one should you actually write? For the exercise itself, Intermediate Approach 2 (try/except around `load_config()`, clean exit, no `main()` yet) is completely correct and what most people should default to — it satisfies "handle the startup failure" without extra machinery. Advanced Approach 2's one-line `sys.exit(message)` is a small, genuinely nicer idiom worth using once you know it. Advanced Approach 3's `main()` wrapper and structured startup logging are worth the extra lines specifically once this script might ever be imported by something else, or once you're running in an environment where logs are collected centrally (a log aggregator, `systemd`, a container platform) and "did this actually start with the config I think it did" is a question you'll really need to answer later.

What carries into the Build Task: these same four files, one level more serious. Copy

`practice/config_logging_wiring/` to `practice/build_task/` and upgrade from there — the [Build Task solution](#) picks up exactly where this leaves off.